

Manual de Usuario — LinkWork

Versión: 1.0 · **Fecha:** 12/08/2026 · **Plataforma:** Web (Flask + MySQL XAMPP)

1. Introducción

LinkWork es una plataforma web que conecta a **empleadores** que buscan mano de obra, **trabajadores (usuarios)** que ofrecen su trabajo y **proveedores de servicios**. Centraliza todo el ciclo laboral: publicar ofertas, postular, conversar, contratar y generar recibos.

¿A quién va dirigido?

- **Usuario (Trabajador):** busca empleo o solicita servicios.
- **Proveedor:** ofrece servicios profesionales (ej. Siembra y Cosecha).
- **Empleador:** publica empleos y gestiona su equipo.
- **Administrador:** supervisa la plataforma.

Objetivo de este manual: guiarte paso a paso para navegar, usar cada pantalla y resolver dudas comunes.

2. Requisitos técnicos

Requisito	Detalle
Navegadores compatibles	Chrome ≥ 80, Firefox ≥ 78, Edge ≥ 85, Safari ≥ 13
Cookies	Activas (necesarias para mantener la sesión <code>session</code> de Flask)
JavaScript	Activado (modales, SweetAlert2, Bootstrap 5)
Pop-ups	Permitidos para modales de chat y formularios
Resolución mínima	1024 × 768 px
Servidor	XAMPP con MySQL (<code>/opt/lampp/var/mysql/mysql.sock</code>) y Python 3 + Flask

3. Acceso y registro

3.1 Página de Introducción

Archivo: Login/intro.html · **Ruta web:** http://127.0.0.1:5000/ (app.py:1522 sirve Login/intro.html)

Es la portada. Muestra la marca LinkWork, el eslogan “Conecta el talento con el trabajo que lo necesita”, el grafo Empleador–Usuario–Proveedor conectados a LW y 3 métricas (3 perfiles, 56h, 100% recibos). Dos botones llevan al login: * **Entrar** (header) → Login/index.html * **Empezar ahora** (hero) → Login/index.html

Incluye toggle claro/oscuro (localStorage lw_tema).

3.2 Login y Registro

Archivo: Login/index.html · **Ruta web:** http://127.0.0.1:5000/Login/ y http://127.0.0.1:5000/Login/index.html (app.py:1536 lo sirve con send_from_directory)

Contiene dos tarjetas conmutables:

a) Iniciar sesión Campos: correo y contraseña. Al enviar, Login/js/app.js llama a POST /api/login . Si las credenciales son válidas, el backend crea sesión y app.js:49 redirige según el rol: * usuario → wed_site/usuario.html * proveedor → wed_site/proveedor.html * empleador → wed_site/empleador.html * admin → wed_site/dashboard.html

b) Registro Campos: nombre de usuario, correo, teléfono, contraseña y rol (desplegable). Valida que el correo/usuario no esté repetido (POST /api/registro → error 400 si existe). Al registrarse, vuelve al login.

Funciones extra en el login: * ¿Olvidaste tu contraseña? Botón .forgot-link → modal recover-qbox con pregunta de seguridad (GET /api/perfil/pregunta). Si responde bien (POST /api/recuperar/cambiar), cambia la contraseña. * Mostrar/ocultar contraseña (.password-toggle , ojos). * Record de sesión bloquea el botón atrás (Login/js/app.js + app.py:1548 no-cache).

4. Navegación y menús

Todas las páginas internas comparten **header superior** (logo LW + top-actions con dark_mode y Cerrar sesión), **sidenav** (offcanvas móvil) y **nav.js** (wed_site/js/nav.js):

- LW.me() verifica sesión; si falla → location.replace('../Login/index.html')

- `logout()` limpia sesión (`POST /api/logout`) y hace `location.replace`
- `pageshow` con `back_forward` cierra sesión automáticamente si se navega atrás
- `Cache-Control: no-store` en HTML evita cache del navegador
- Inyecta modal de perfil y tema (`lw_tema`)

Mapa del sitio:

```
/ (Login/intro.html) → /Login/ (Login/index.html) → /wed_site/{usuario.html|profile.html}
|-> css/ , js/ , Login/js/api.js
```

Buscador interno: cada sección tiene `<input oninput="filtrarLista('myJobs', this.value)">` (`Login/js/api.js:29`).

5. Descripción detallada de cada HTML

5.1 `Login/intro.html` — Introducción / Portada

Rol: Público (sin login) **Secciones:** * `header.top` — logo LW, toggle tema, Entrar * `section.hero` — badge “Empleo · Servicios · Contratos”, H1 con degradado en *trabajo*, párrafo lead, dos CTAs, **hero-graph** SVG (triángulo Empleador–Usuario–Proveedor todos conectados a LW + 3 líneas al centro), **hero-stats** (3, 56h, 100%) * `footer.intro-footer` — pie

Función: explicar qué es LinkWork y llevar al login. Su CSS está en `Login/css/style.css` bajo `body.intro-page` (no afecta al login).

5.2 `Login/index.html` — Autenticación

Rol: Público **Estructura:** `head` con Inter + Material Icons + `Login/css/style.css` , `body` con partículas flotantes, `brand-header` (logo grande), `auth-toggle` (dos tabs), `card` con dos formularios: * `#loginCard` (email, password, forgot-link, btn Entrar) * `#registerCard` (username, email, phone, password, select rol)

Scripts: `Login/js/api.js` (función `api()`) y `Login/js/app.js` (lógica de login/registro/recuperación).

5.3 `wed_site/dashboard.html` — Panel Administrador

Rol: admin · **Ruta:** `GET /api/usuarios` (solo admin) **Secciones (`data-section`):** * `general` — 4 stats: usuarios totales, empleos, servicios, contratos y aplicaciones pendientes (`/api/finanzas/resumen` no, sino conteos directos) * `usuarios` — tabla de usuarios con rol y frecuencia, botón Eliminar (`DELETE /api/usuarios/<email>`) * `empleos` — lista de todos

los empleos * servicios — lista de todos los servicios **Navegación:** dd-section-trigger conmuta section-content .

5.4 wed_site/empleador.html — Panel Empleador

Rol: empleador **Secciones:** * publicar — form#jobForm con: jobTitle , jobCompany , jobLocation , jobSalary , jobCupos , jobTipo (fijo/hora), jobFormalidad (formal/informal), jobHoraEntrada/jobHoraSalida , jobHoras , jobDescription , jobImagen . Envía POST /api/empleos (formalidad incluida). * misempleos — lista myJobs con badge cupos_ocupados/cupos y formalidad (aceptado=Formal, pendiente=Informal), botones Editar(modal ajusteEmpleoModal con ajusteFormalidad) → PATCH /api/empleos/<id> y Eliminar → DELETE /api/empleos/<id> * solicitudes — solicitudesList con aplicaciones tipo=empleo y estado=pendiente , botones Perfil (openPerfilPublico), Chat (openChatTab), Aceptar (pide monto → PATCH /api/aplicaciones/<id>/estado=aceptado + POST /api/contratos) y Rechazar * servicios — empleadorServList lista servicios públicos, botones Solicitar (openApplyModal) y Contactar + Perfil (perfil-serv-btn → openPerfilPublico) * empresa — **empresaEmpleadorList** :hero empresa(/api/perfil/empresa), stats (contratos activos, equipo, nómina, equipos por puesto) y **Trabajos de mi equipo** (emp-group por ref_titulo): cada emp-team-item con avatar, nombre, email, monto, días, **estado del compañero** (estadoMap de GET /solicitudes/recibidas con aceptado → De Vacaciones/De Permiso/Activo) y botones Configurar, Recibo, Finalizar (PATCH /api/contratos/<id>), Chat, Perfil. Estado calculado por _estado_trabajo(email) si hay solicitud aceptada. * chat y grupo — chatForm → POST /api/mensajes y POST /api/grupo/<id>/mensajes

5.5 wed_site/proveedor.html — Panel Proveedor

Rol: proveedor **Secciones:** * publicar — formulario de servicio: sTitle , sCategory , sPrice , sDuracion , sUbicacion , sHorarios , sIncluye , sImagen → POST /api/servicios * servicios — misServicios lista con Configurar (abrirConfigServicio → PATCH /api/servicios/<id>) y Eliminar, muestra imagen (banner listing-img 180px), chips horarios / incluye * solicitantes — lista de aplicaciones tipo=servicio con Chat (accent), Aceptar (success), Perfil (violet), Rechazar * empleos — provJobList lista empleos para Contactar + Perfil (perfil-btn) * ingresos — gráfico finChart con GET /api/finanzas * frecuentes — frecuentesList con GET /api/frecuentes (solo usuarios que solicitaron **tus** servicios, calculado por COUNT(a.id) donde propietario_email=tu email → ya no global) * chat y grupo — idem empleador

5.6 wed_site/usuario.html — Panel Usuario

Rol: usuario **Secciones:** * empleos — jobList con renderJobs() :badge services/jobs , listing-img si hay imagen , listing-details (ubicación, salario, Por Hora/Fijo, **Formal/Informal**, horas , cupos restantes), descripción, fav-btn (toggleFav), apply-btn

→ openApplyModal y chat-listing-btn + perfil-listing-btn (ambos si no es tu propia oferta) * servicios — serviceList idem, con duracion , ubicacion , Solicitar + Chat + Perfil * postulaciones — postulacionesList con estado pendiente/ aceptado/rechazado , datos del contrato si existe y botón Chat * recomendaciones — recomendacionesList con aviso 48–56h * empresa — miEmpresaListo / **empresaResumen** : GET /api/mi-empresa trae contrato (con dias_trabajados por registrar_dias_automaticos), empresa y **compañeros con estado_trabajo** (_estado_trabajo backend: busca última solicitudes aceptada del compañero → vacaciones/permiso/activo). Muestra emp-hero , chips de contrato, emp-team-item por compañero con badge de estado (info=Vacaciones, warning=Permiso, success=Activo). Además empSolicitudBtn → POST /api/solicitudes (vacaciones 1-15 días, permiso 1-3 días, una vacaciones/año). * chat / grupo — idem

Nota común: Todas las tarjetas listing-card tienen barra izquierda ::before de 4px que al hover se anima (transform: scaleY(1)) con background: var(--text) (negra en claro, blanca en oscuro).

6. Gestión de contenidos (Administrador)

Solo admin@admin.com (hash admin123) ve dashboard.html : * **Ver usuarios:** GET /api/usuarios → tabla, rol, frecuencia * **Eliminar usuario/servicio/empleo:** DELETE /api/usuarios/<email> y DELETE en empleos/servicios * **No hay subida de textos genéricos:** cada rol gestiona su propio contenido (empleador publica empleos, proveedor servicios). El admin no edita textos globales. * **Crear admin:** init_db() inserta admin@admin.com si no existe.

7. Preguntas frecuentes y soporte

Problema	Causa	Solución
"El servidor respondió con datos inválidos (código 415)" al finalizar contrato	PATCH /api/contratos/<id> sin body	En Login/js/api.js:142 ahora envía {estado:'finalizado'}
"Ya te has postulado a esto" en primer intento	Doble clic rápido → 2º request ve la 1º pendiente	applySubmitBtn se deshabilita (btn.disabled) durante LW.aplicar() en

Problema	Causa	Solución
		usuario.html:288 y empleador.html:894
No deja enviar mensajes (500)	destinatario_nombre llegaba NULL (chat desde Postulaciones)	app.py:1361 busca el username por id / email si falta
"Data truncated for column 'tipo_ref'" al chatear	mensajes.tipo_ref era ENUM('empleo','servicio') pero se envía general / grupo	ALTER TABLE mensajes MODIFY tipo_ref VARCHAR(50) DEFAULT 'general'
Yarleydy ve usuarios frecuentes sin solicitudes	GET /api/frecuentes devolvía frecuencia global	Ahora GROUP BY solo donde propietario_email=tu email y excluye tu propio email
Cupos muestran 9 cuando deberían ser 10	cupos_ocupados contaba aceptados con contrato finalizado	Ahora NOT EXISTS (contrato finalizado) → al finalizar libera cupo
No puedo volver a postular tras rechazo	POST /api/aplicaciones bloqueaba si existía cualquier previa	Ahora solo bloquea si hay pendiente o aceptado con contrato activo; permite tras rechazado o aceptado finalizado. Además valida 56h semanales y ya contratado en la empresa.
"No autorizado" al navegar atrás	Back mostraba página cacheada	Cache-Control: no-store + pageshow back_forward → LW.logout() + location.replace
database.db aparece y baja calificación	Proyecto debe usar MySQL XAMPP, no SQLite	app.py:19 DB_CONFIG con unix_socket: /opt/lampp/ var/mysql/mysql.sock y database: Linkwork ; database.db eliminado y añadido a .gitignore
renderCurrentSection is not defined	Variable sin declarar	let renderCurrentSection = null en

Problema	Causa	Solución
		usuario.html:264 y empleador.html:371

Datos de contacto: No hay soporte externo configurado. Contactar al administrador (admin@admin.com) o revisar logs en /tmp/opencode/linkwork.log . Los mensajes de error se muestran con SweetAlert2 (swalErr).

8. Anexo: Estructura de archivos HTML del proyecto

Proyecto Final LP,	
└ Login/	
├ intro.html	← Portada pública, grafo Empleado-Usuario-Proveedor,
├ index.html	← Login/Registro/Recuperación (auth-toggle, recover-qb)
├ css/style.css	← Estilos login + intro (hero, top, blueprint, dashflo)
└ js/app.js + js/api.js	← api() con ?_timestamp, LW.logout/me/empleos/servi
└ web site/	
├ css/style.css	← Estilos globales (listing-card, emp-*, chat, dark th)
├ js/nav.js	← Nav, inyecta perfil, tema lw tema, notificaciones, o
├ usuario.html	← 6 secciones usuario (ver \$5.6)
├ proveedor.html	← 7 secciones proveedor (ver \$5.5)
├ empleador.html	← 6 secciones empleador (ver \$5.4)
└ dashboard.html	← Panel admin (ver \$5.3)
└ docs/	
├ diagrama flujo.html	← Un solo flowchart Mermaid (Empleo, Servicio, Reglas,
└ DIAGRAMA FLUJO.md	← Mismo diagrama en markdown
└ uploads/	← Imágenes subidas (POST /api/upload)
└ app.py	← Flask 5000, MySQL XAMPP, 30+ rutas /api/*
└ .gitignore	← ignora database.db
└ MANUAL_USUARIO.md/pdf	← Este manual

Cada HTML es una **SPA por secciones** (section-content con display:none y dd-section-trigger conmuta). Ningún HTML recarga la página; todo es fetch a /api/* con credentials: include .

Fin del manual. Para regenerar el PDF: `python3 -c "import markdown, weasyprint; html=markdown.markdown(open('MANUAL_USUARIO.md').read()); weasyprint.HTML(string=html).write_pdf('MANUAL_USUARIO.pdf')"`